Dealing with Errors in Power BI

Introduction

Power BI Desktop is the free tool from Microsoft for developing Business Report in Power BI. This tutorial walks you through importing and manipulating data with the Query Editor in Power BI Desktop.

This tutorial focuses specifically on Power BI Desktop because of the important role it plays in creating reports that can then be used by the other components.

The features in Power BI Desktop are organized into three views that you access from the navigation pane at the left side of the main window:

- **Report view:** A canvas for building and viewing reports based on the datasets defined in *Data* view.
- **Data view:** Defined datasets based on data retrieved from one or more data sources. *Data* view offers limited transformation features, with many more capabilities available through Query Editor, which opens in a separate window.
- **Relationships view:** Identified relationships between the datasets defined in *Data* view. When possible, Power BI Desktop identifies the relationships automatically, but you can also define them manually.

To access any of the three views, click the applicable button in the left navigation pane.

When working in Power BI Desktop, you will normally perform the following four basic steps, although not necessarily in a single session:

1. Connect to one or more data sources and retrieve the necessary data.
2. Transform and enhance the data using *Data* view, *Relationships* view, or the Query Editor, as necessary.
3. Create reports based on the transformed data, using *Report* view.
4. Publish the reports to the Power BI service or upload them to Power BI Report Server.

This tutorial will focus primarily on steps 1 and 2 so you can get a better sense of how to retrieve and prepare the data, which is essential to building comprehensive reports. Later in the series, you'll learn more about building reports with different types of visualizations, but first you need to make sure you get the data right.

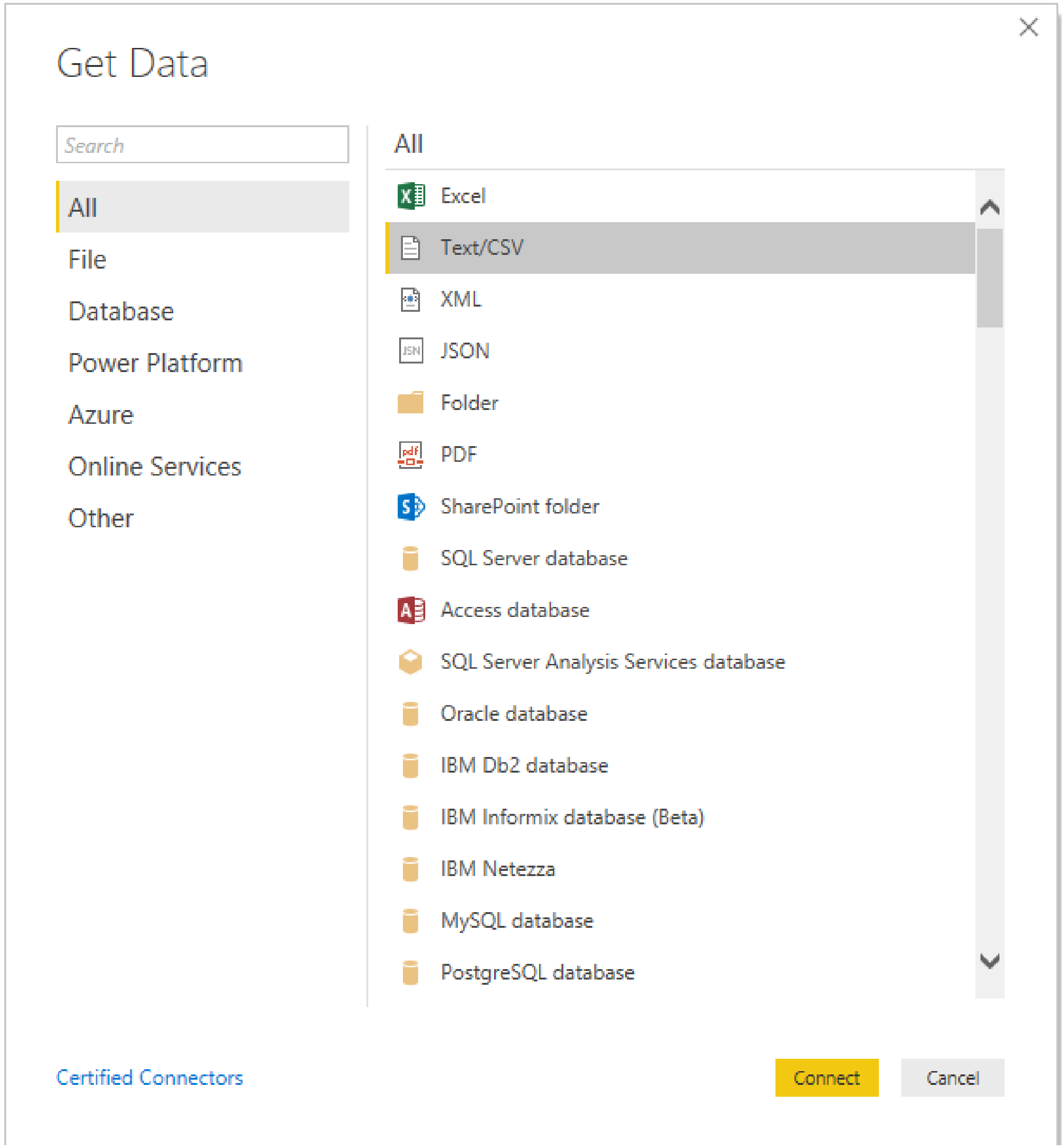
Step 1: Connecting to Data in Power BI Desktop

This tutorial uses a CSV file based on data from the Hawks dataset. This is available on Blackboard as **Power BI – Task 1.csv** but was originally obtained from through a GitHub collection of sample datasets. The dataset contains data collected over several years from a hawk blind at Lake MacBride near Iowa City, Iowa. Download and save the file to a local folder (e.g. ***C:\DataFiles*** *or Desktop*). From now on, every time I'll refer to ***C:\DataFiles***, you'll have to change this as the path to the folder where you saved your file (it might be something like U:\MyFiles\......).

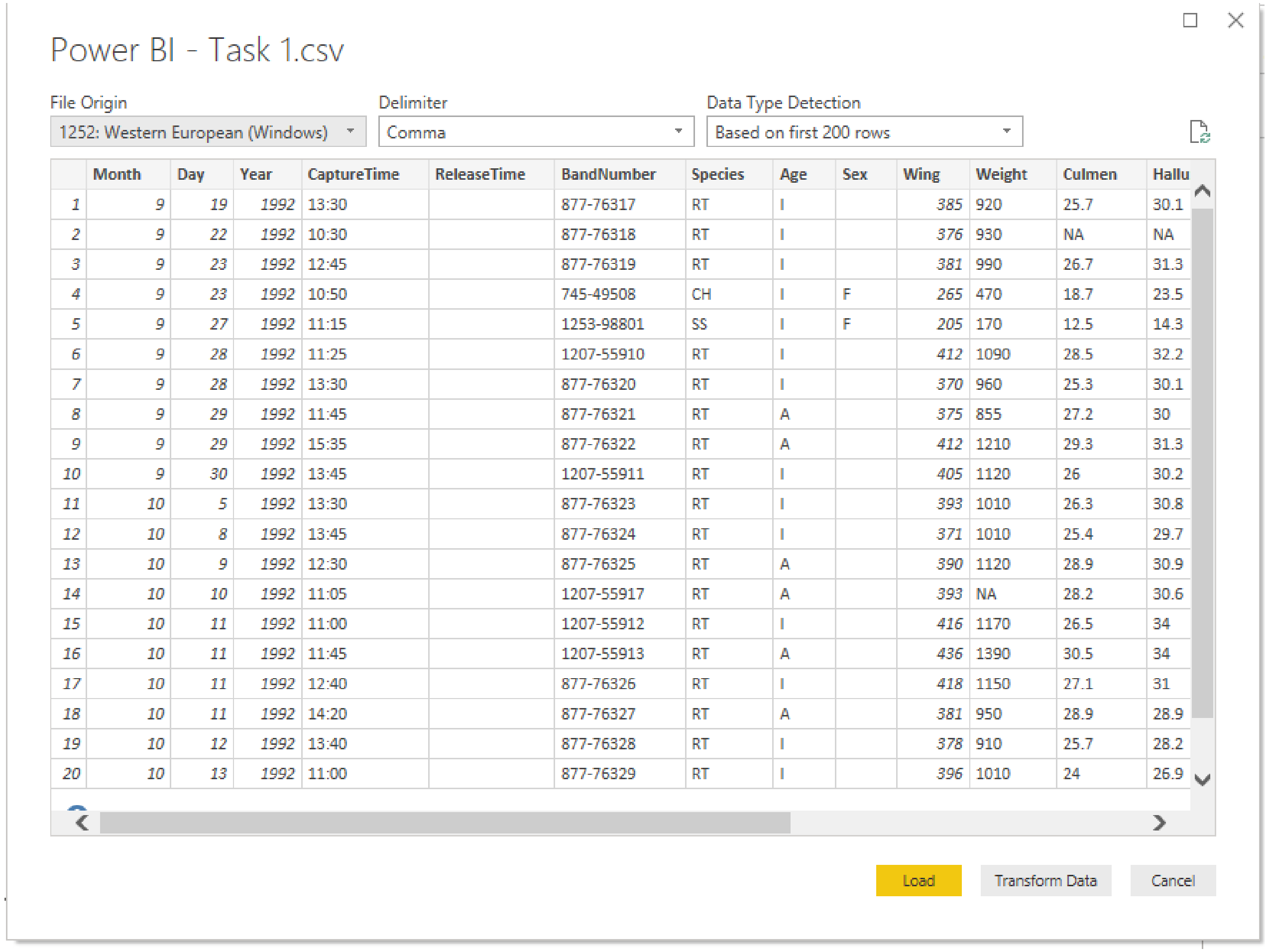
Power BI Desktop supports connectivity to a wide range of data sources, which are divided into the following categories:

- **All:** Every data source type available through Power BI.
- **File:** Source files such as Excel, CSV, XML, or JSON.
- **Database:** Database systems such as SQL Server, Oracle Database, IBM DB2, MySQL, SAP HANA, and Amazon Redshift.
- **Azure:** Azure services such as SQL Database, SQL Data Warehouse, Blob Storage, and Data Lake Store.
- **Online Services:** Non-Azure services such as Google Analytics, Salesforce Reports, Facebook, Microsoft Exchange Online, and the Power BI service.
- **Other:** Miscellaneous data source types such as Microsoft Exchange, Active Directory, Hadoop File System, ODBC, OLE DB, and OData Feed.

To retrieve data from one of these data sources, click the *Get Data* button on the *Home* ribbon in the main Power BI window. This launches the *Get Data* dialog box, shown in the following figure.


Get Data
Search
All
File
Database
Power Platform
Azure
Online Services
Other
All
Excel
Text/CSV
XML
JSON
Folder
PDF
SharePoint folder
SQL Server database
Access database
SQL Server Analysis Services database
Oracle database
IBM Db2 database
IBM Informix database (Beta)
IBM Netezza
MySQL database
PostgreSQL database
Certified Connectors
Connect
Cancel

To import the data from the file into Power BI Desktop, select the *Text/CSV* data source type in the *Get Data* dialog box. Click *Connect* and the *Open* dialog box will appear. From there, navigate to the ***C:\DataFiles*** folder (or Desktop folder), select the ***Power BI - Task 1.csv*** file, and click *Open.* This launches the preview window shown in the following figure.



	Month	Day	Year	CaptureTime	ReleaseTime	BandNumber	Species	Age	Sex	Wing	Weight	Culmen	Hallu
1	9	19	1992	13:30		877-76317	RT	I		385	920	25.7	30.1
2	9	22	1992	10:30		877-76318	RT	I		376	930	NA	NA
3	9	23	1992	12:45		877-76319	RT	I		381	990	26.7	31.3
4	9	23	1992	10:50		745-49508	CH	I	F	265	470	18.7	23.5
5	9	27	1992	11:15		1253-98801	SS	I	F	205	170	12.5	14.3
6	9	28	1992	11:25		1207-55910	RT	I		412	1090	28.5	32.2
7	9	28	1992	13:30		877-76320	RT	I		370	960	25.3	30.1
8	9	29	1992	11:45		877-76321	RT	A		375	855	27.2	30
9	9	29	1992	15:35		877-76322	RT	A		412	1210	29.3	31.3
10	9	30	1992	13:45		1207-55911	RT	I		405	1120	26	30.2
11	10	5	1992	13:30		877-76323	RT	I		393	1010	26.3	30.8
12	10	8	1992	13:45		877-76324	RT	I		371	1010	25.4	29.7
13	10	9	1992	12:30		877-76325	RT	A		390	1120	28.9	30.9
14	10	10	1992	11:05		1207-55917	RT	A		393	NA	28.2	30.6
15	10	11	1992	11:00		1207-55912	RT	I		416	1170	26.5	34
16	10	11	1992	11:45		1207-55913	RT	A		436	1390	30.5	34
17	10	11	1992	12:40		877-76326	RT	I		418	1150	27.1	31
18	10	11	1992	14:20		877-76327	RT	A		381	950	28.9	28.9
19	10	12	1992	13:40		877-76328	RT	I		378	910	25.7	28.2
20	10	13	1992	11:00		877-76329	RT	I		396	1010	24	26.9

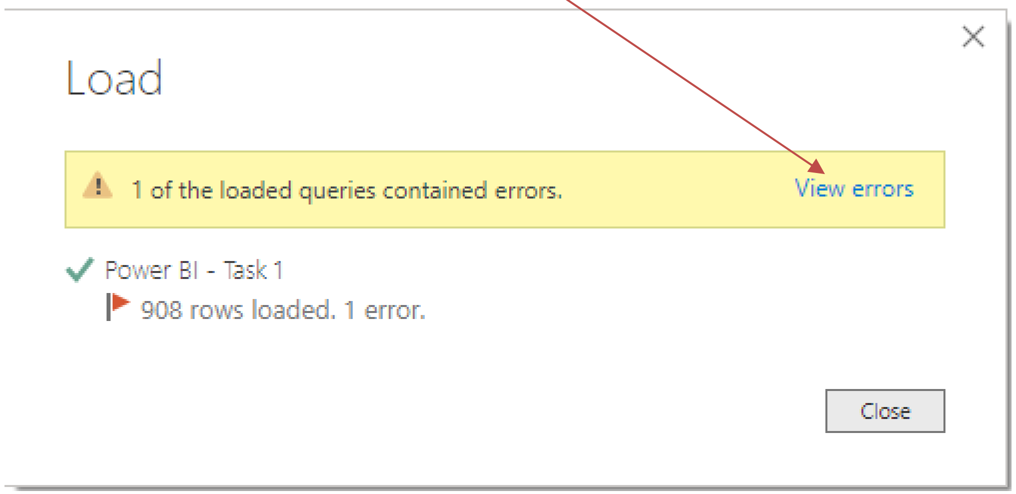
In addition to being able to preview a subset of the data, you can configure several options: *File Origin* (the document's encoding), *Delimiter,* and *Data Type Detection*. In most cases, you will be concerned primarily with the *Data Type Detection* option. By default, this is set to *Based on first 200 rows,* which means that Power BI Desktop will convert the data to Power BI data types based only on the first 200 rows of data.

This might be okay in some cases, but there could be times when a column includes a value different from the majority of values, but that value is not in the first 200 rows. As a result, you could end up with a suboptimal data type or even an error. To avoid this risk, you can use the *Data Type Detection* option to instruct Power BI Desktop to base the data type selection on the entire dataset, or you can instead forego any conversion and keep all the data as string values.

You could also edit the dataset before loading it into Power BI Desktop (we are not doing this for now). But, if you click the *"Transform your data"* button, Power BI Desktop launches another window, called Query Editor, which provides a number of tools for transforming data. In this way, after you make the changes, you can then save the updated dataset.

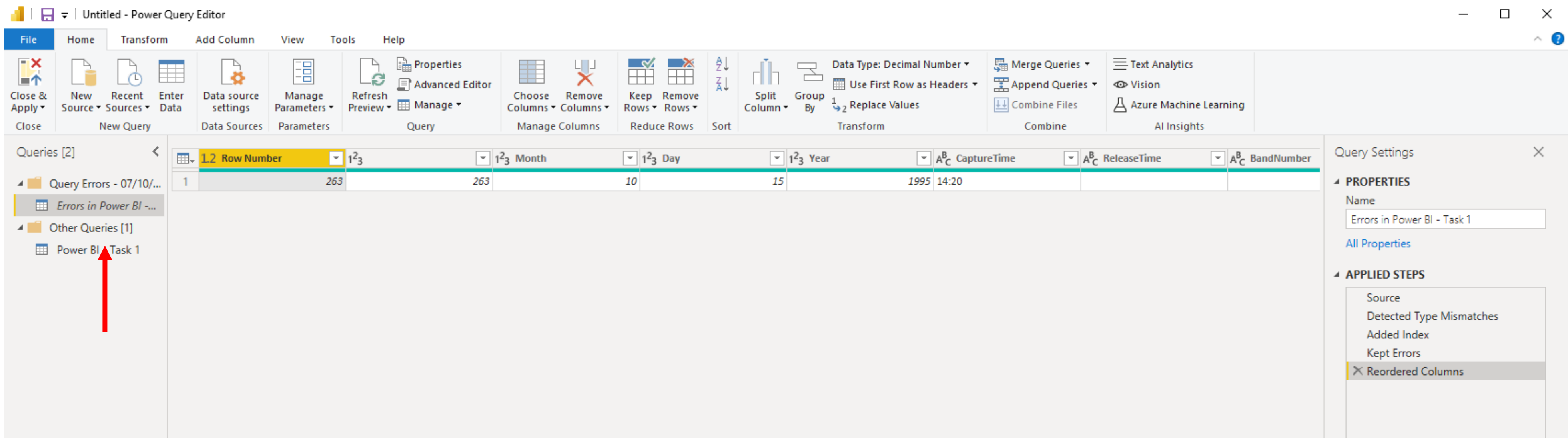
In this case, load the data into Power BI Desktop without changing any settings or editing the dataset. You will be transforming the data later as part of working through this tutorial. To import the data as it is, you need only click the *Load* button.

If you'll see a window like the one below, click **View errors**



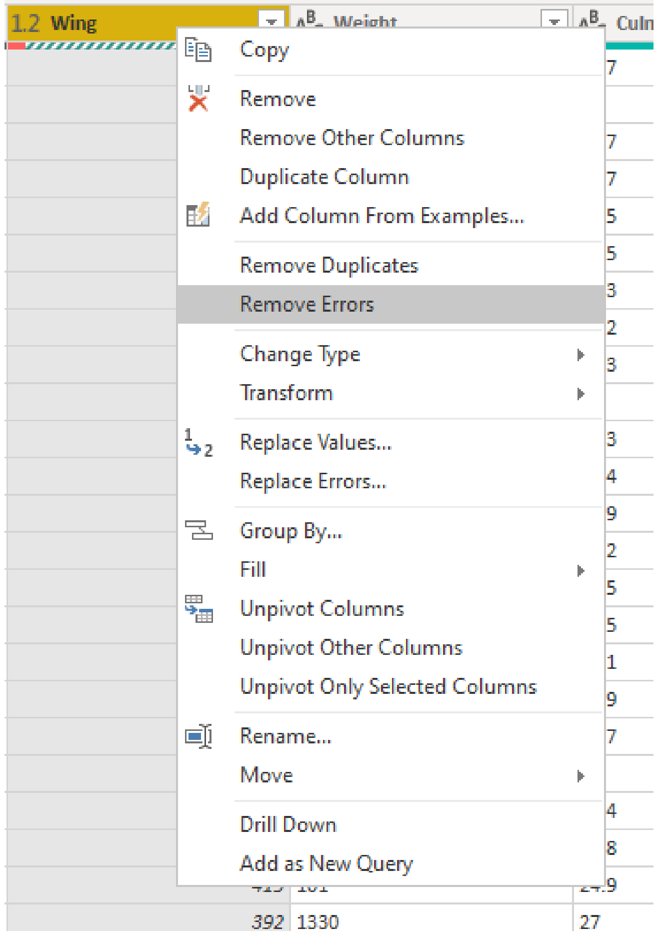
Step 2: Addressing Errors

When you click the *View errors* link, Query Editor will open and isolate the row that contains the error, as shown in the following figure. The *Row Number* value for this row is *263*. After viewing the error row, you should see that the *Wing* column contains an *NA* value, although the column is configured with the **number** data type, which is why you received the error.



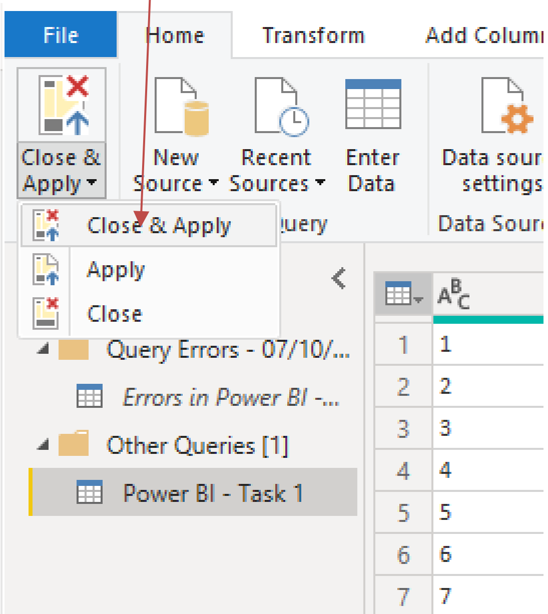
Click "Power BI – Task 1" on the left to exit out of the *Errors* screen and get back to the Query Editor.

To address the error in Query Editor right-click the column header of the Wing column, click *Remove Errors.*

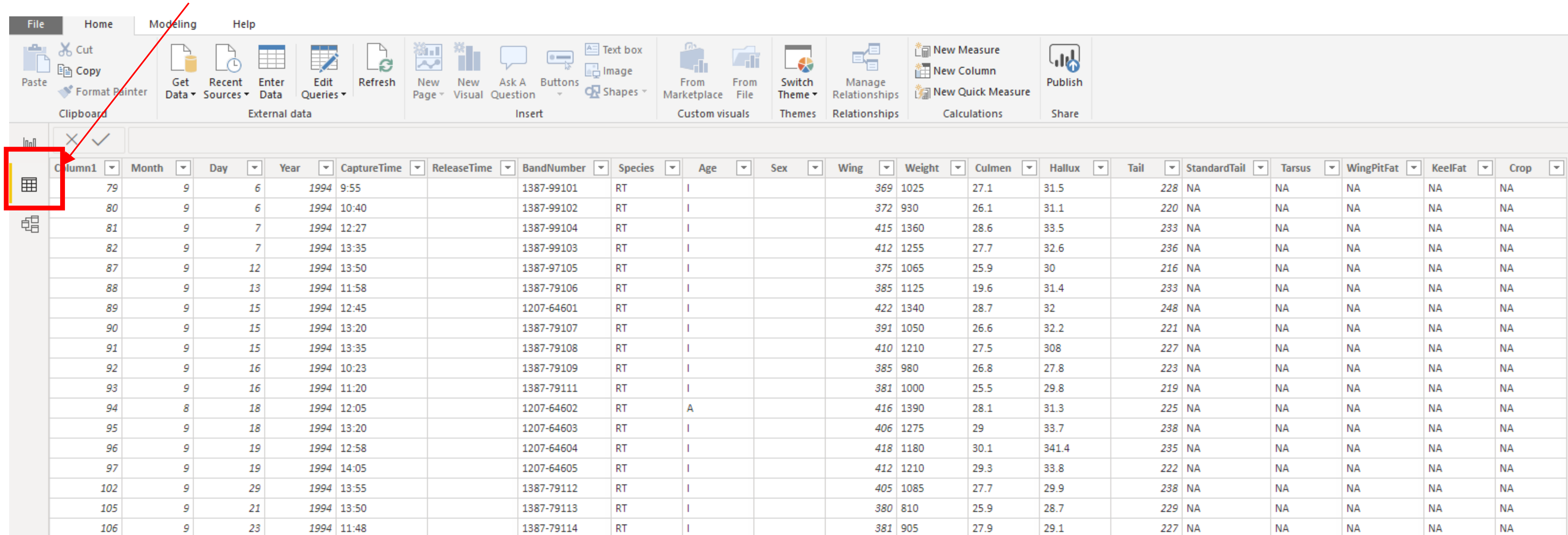


NOTE: One thing interesting about the error is that it occurs as a result of promoting the first row to column headers and the subsequent type conversion that came with it. From what I can tell, Query Editor must use only a sample of the data when selecting the data type during this step. It does not appear to have anything to do with the Data Type Detection setting in the preview window. I tried all possible settings and the results were always the same.

Click Close&Apply to apply these changes.



You can now view the imported dataset in *Data* view, as shown in the following figure.

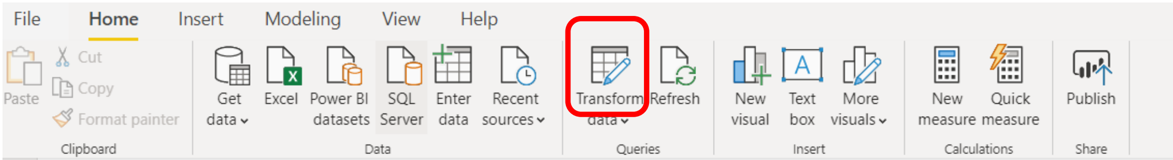


Column1	Month	Day	Year	CaptureTime	ReleaseTime	BandNumber	Species	Age	Sex	Wing	Weight	Culmen	Hallux	Tail	StandardTail	Tarsus	WingPitFat	KeelFat	Crop
79	9	6	1994	9:55		1387-99101	RT	I		369	1025	27.1	31.5	228	NA	NA	NA	NA	NA
80	9	6	1994	10:40		1387-99102	RT	I		372	930	26.1	31.1	220	NA	NA	NA	NA	NA
81	9	7	1994	12:27		1387-99104	RT	I		415	1360	28.6	33.5	233	NA	NA	NA	NA	NA
82	9	7	1994	13:35		1387-99103	RT	I		412	1255	27.7	32.6	236	NA	NA	NA	NA	NA
87	9	12	1994	13:50		1387-97105	RT	I		375	1065	25.9	30	216	NA	NA	NA	NA	NA
88	9	13	1994	11:58		1387-79106	RT	I		385	1125	19.6	31.4	233	NA	NA	NA	NA	NA
89	9	15	1994	12:45		1207-64601	RT	I		422	1340	28.7	32	248	NA	NA	NA	NA	NA
90	9	15	1994	13:20		1387-79107	RT	I		391	1050	26.6	32.2	221	NA	NA	NA	NA	NA
91	9	15	1994	13:35		1387-79108	RT	I		410	1210	27.5	308	227	NA	NA	NA	NA	NA
92	9	16	1994	10:23		1387-79109	RT	I		385	980	26.8	27.8	223	NA	NA	NA	NA	NA
93	9	16	1994	11:20		1387-79111	RT	I		381	1000	25.5	29.8	219	NA	NA	NA	NA	NA
94	8	18	1994	12:05		1207-64602	RT	A		416	1390	28.1	31.3	225	NA	NA	NA	NA	NA
95	9	18	1994	13:20		1207-64603	RT	I		406	1275	29	33.7	238	NA	NA	NA	NA	NA
96	9	19	1994	12:58		1207-64604	RT	I		418	1180	30.1	341.4	235	NA	NA	NA	NA	NA
97	9	19	1994	14:05		1207-64605	RT	I		412	1210	29.3	33.8	222	NA	NA	NA	NA	NA
102	9	29	1994	13:55		1387-79112	RT	I		405	1085	27.7	29.9	238	NA	NA	NA	NA	NA
105	9	21	1994	13:50		1387-79113	RT	I		380	810	25.9	28.7	229	NA	NA	NA	NA	NA
106	9	23	1994	11:48		1387-79114	RT	I		381	905	27.9	29.1	227	NA	NA	NA	NA	NA

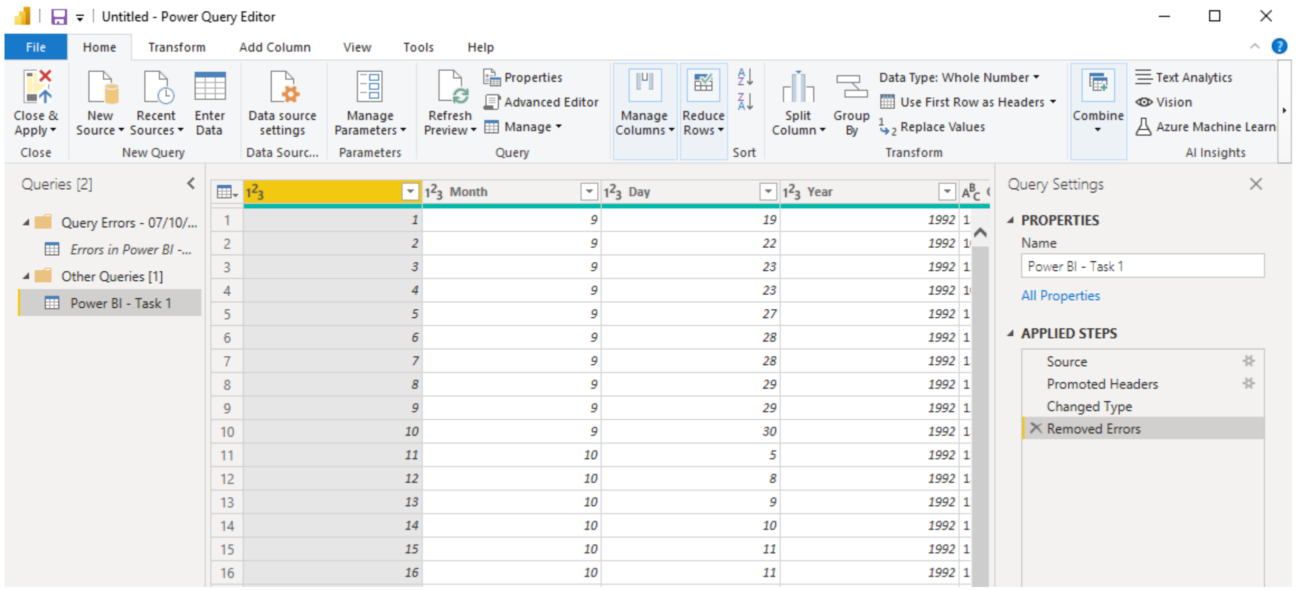
After you import a dataset in Power BI Desktop, you can use the data in your reports as it is, or you can edit the data, applying various transformations and in other ways shaping the data. Although you can make some modifications in *Data* view, such as being able to add or remove columns or sort data, for the majority of transformations, you will need to use Query Editor, which includes a wide range of features for shaping data.

Step 3: Modifying Your Data with the Query Editor

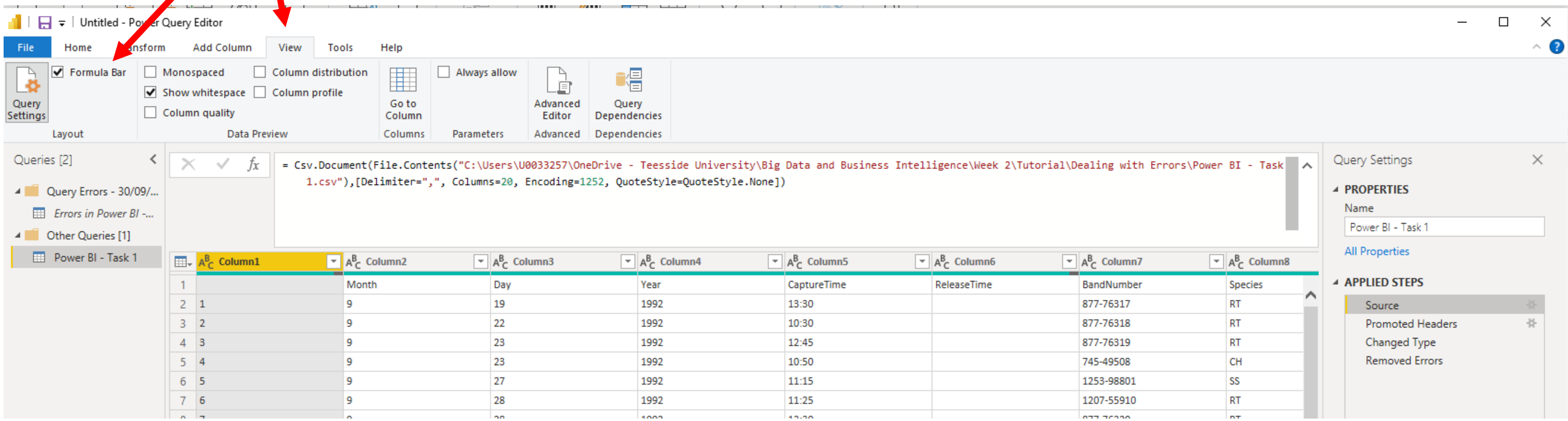
To launch again the Query Editor, click the Transform Data button on the *Home* ribbon.



Query Editor opens as a separate window from the main Power BI Desktop window and displays the contents of the dataset selected in the left pane, as shown in the following figure. In this case, the pane includes only the *Power BI – Task 1* dataset, which is the one you just imported. The dataset itself is displayed in the main pane of the Query Editor window. You can scroll up and down or right and left to see all the data. The pane to the right, which is separated into two sections, provides additional information about the dataset. The *Properties* section displays the dataset's properties. In this case, the section includes only the *Name* property. To view all properties associated with a dataset, click the *All Properties* link within that section.



The *Applied Steps* section lists the transformations that have been applied to the dataset in the order they were applied. Each step in this section is associated with a script statement written in the Power Query M formula language. The statement is shown in the formula bar directly above the dataset. (If the formula bar is not displayed, select the *Formula Bar* checkbox on the *View* ribbon to display it, as shown below.)



Currently, the *Applied Steps* section for the dataset includes only four steps: *Source, Promoted Headers*, *Changed Type and Removed Errors.* These steps are generated by default when importing a CSV file. Click on the *Source* step and you will see that this corresponds to the following M statement in the formula bar:

= Csv.Document(File.Contents("C:\DataFiles\Power BI – task 1.csv"),[Delimiter=",", Columns=20, Encoding=1252, QuoteStyle=QuoteStyle.None])

If you are not familiar with coding, the M Formula language might look complicated. Don't worry about it, we will cover this too in a later session of the module.

The M statement consists primarily of the connection string necessary to import the *hawks.csv* file. Notice that it includes the delimiter and encoding specified in the preview window, along with other details. If you click on the other Steps in the applied Steps pane, you can visualize the corresponding M formulae.

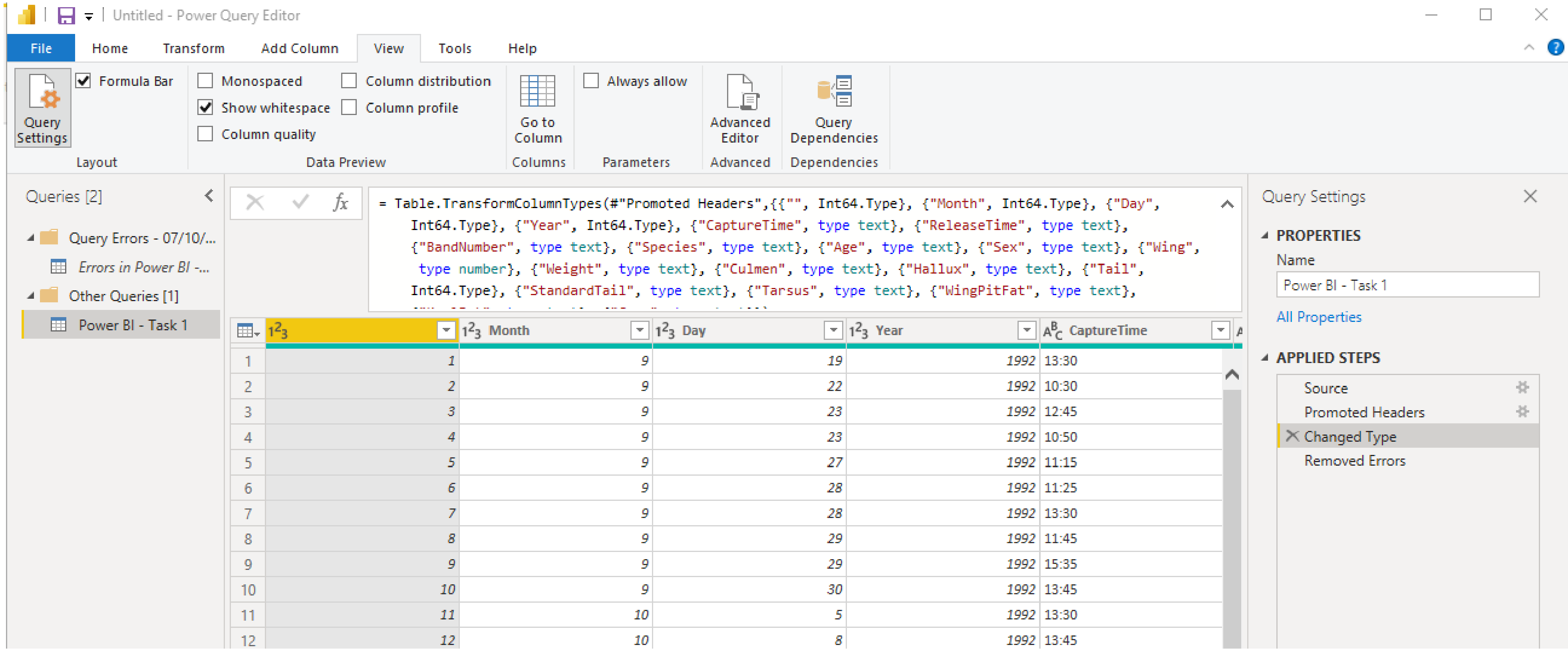
The "Promoted Headers" step, this is to promote the first row of values as the new column headers (i.e. column names). By default, only text or number values are promoted to headers. Here is the associated M formula:

= Table.PromoteHeaders(Source, [PromoteAllScalars=true])

The *Changed Type* step converts the data in each column to Power BI data types. The step is associated with the following M statement, which shows that the first column (which currently has no name – this is why there is an empty string "" here) is assigned the **Int64** type (whole number) and that all other columns are assigned the **text** type:

= Table.TransformColumnTypes(#"Promoted Headers",{{"", Int64.Type}, {"Month", Int64.Type}, {"Day", Int64.Type}, {"Year", Int64.Type}, {"CaptureTime", type text}, {"ReleaseTime", type text}, {"BandNumber", type text}, {"Species", type text}, {"Age", type text}, {"Sex", type text}, {"Wing", type number}, {"Weight", type text}, {"Culmen", type text}, {"Hallux", type text}, {"Tail", Int64.Type}, {"StandardTail", type text}, {"Tarsus", type text}, {"WingPitFat", type text}, {"KeelFat", type text}, {"Crop", type text}})

Notice that, in addition to the first column, several other columns have been converted to the **Int64** data type (whole number - type). However, the *Wing* column has been converted to the **number** type (decimal number - type) because it contains at least one decimal value. Other columns retained the **text** data type because they contained only character values or contained a mix of types. If you had opted to forego data type conversions when importing the file, all columns would be assigned the **text** data type, but you instead stuck with the *Based on first 200 rows* option, which resulted in one column being converted to the **Int64** type.



This tutorial won't spend much time on the M language. Just know that the M language is the driving force behind the Power BI Desktop transformations in Query Editor. Tutorials later in the module will dig into the language in more detail so you can work with it directly.

Every transformation you apply to a dataset is listed in the *Applied Steps* section, and each step is associated with an M statement that builds on the preceding step. You can use the steps and their associated M statements to work with the dataset in various ways. For example, if you select a step in the *Applied Steps* section, Query Editor will display the dataset as it existed when the step was applied. You can also modify steps, delete steps, inject steps in between others, or move steps up or down. In fact, the *Applied Steps* section is one of the most powerful tools you have for working with Power BI Desktop datasets. Be aware, however, that it's very easy to introduce an error into the steps that can turns all your work into a query-error.

Step 4: Renaming a column and Removing Columns

Double click on the first column and rename it as "TagID".

	TagID	Month
1	1	9
2	2	9
3	3	9
4	4	9
5	5	9
6	6	9
7	7	9
8	8	9

In some cases, the data you import includes columns that you do not need for your reports. Query Editor makes removing these columns a simple and quick process.

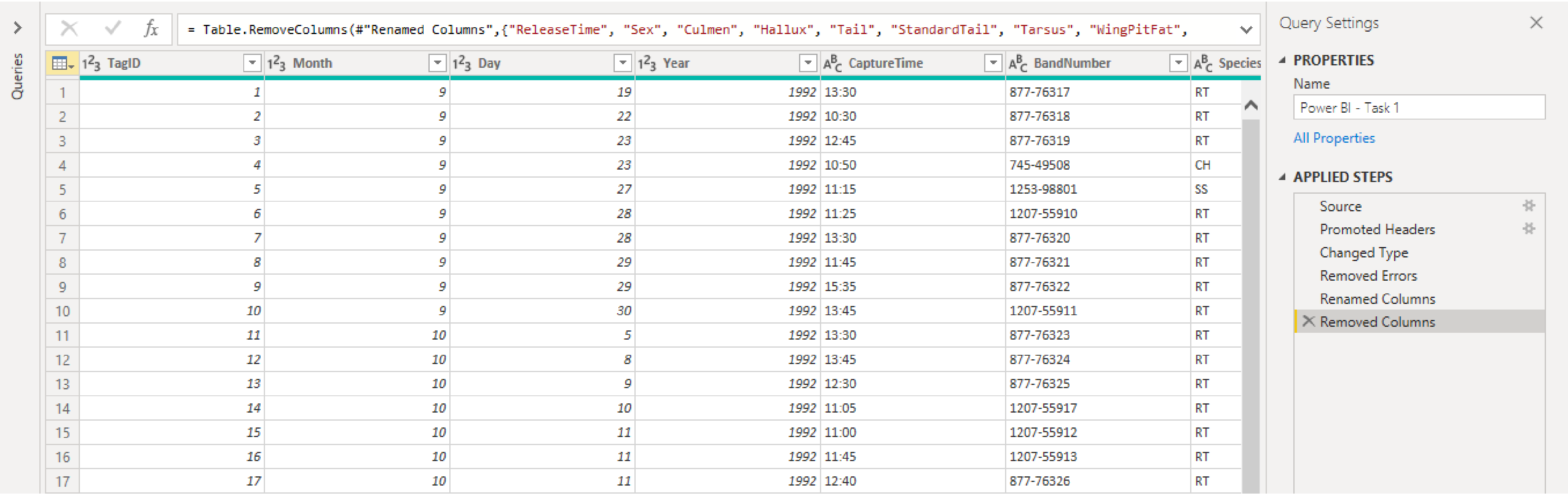
- To remove a single column, select the column directly in the displayed dataset, and then click the *Remove Columns* button on the *Home* ribbon, or you can instead right-click the column header and then click *Remove*.
- To remove multiple columns, select the first column, press and hold the *Control* key, select each additional column, and then click the *Remove Columns* button on the *Home* ribbon.

For this dataset, remove the following columns because they are not relevant to the reports you might create or they contained numerous *NA* values:

- ReleaseTime
- Sex

- Culmen
- Hallux
- Tail
- StandardTail
- Tarsus
- WingPitFat
- KeelFat
- Crop

After removing the columns, Query Editor adds a single *Removed Columns* step to the *Applied Steps* section, as shown in the following figure.

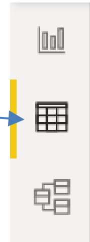


One of the advantages of having the *Removed Columns* step listed along with the other steps is that you can easily remove or modify the step if you later decide that one or more of the columns should not have been removed. In this case, you're relatively safe modifying this step because any steps that you might have added after deleting the columns cannot reference those columns because they're considered nonexistent. Close the Query Editor by clicking **Close & Apply** in the top left corner to go back to *Data* view.

Step 5: Adding a Calculated Column

With Power BI Desktop, you can add calculated columns to a dataset that concatenates data or performs calculations. Power BI Desktop provides two methods for adding a calculated column. The first is to create the column in *Data* view, using the Data Analysis Expressions (DAX) language to define the column's logic. In this tutorial, you will use just one DAX formula. Later on, you will have a whole lesson on DAX.

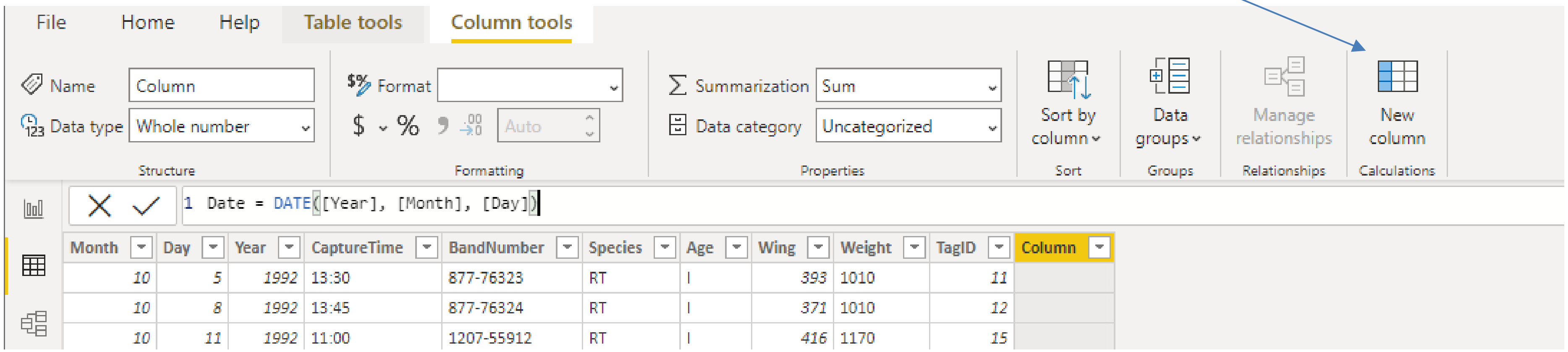
To create a DAX-based column, open the Data view (second tab in the left menu),



then click *New Column* button in the Column Tools ribbon. Then type the DAX expression in the formula bar at the top of the dataset (see image below).

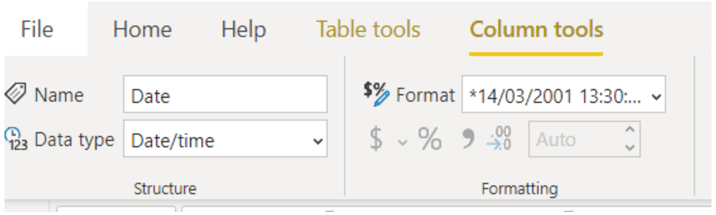
The new column should be selected when adding the expression. After you type the expression, click the checkmark to the left of the expression to verify the syntax and populate the new column.

For example, add a column named *Date* to provide a single value that shows the date when a hawk was tagged, using the following DAX expression:

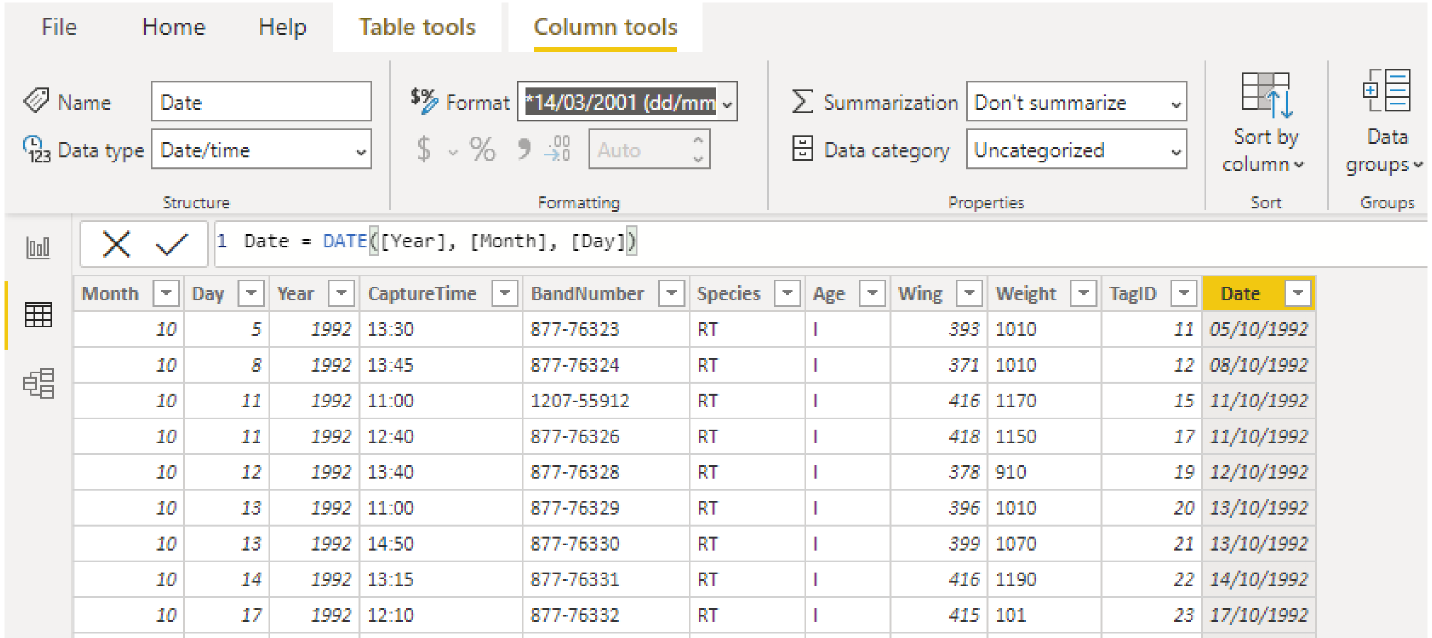


Date = DATE([Year], [Month], [Day])

The expression uses the **DATE** function to create a date value based on the *Year, Month,* and *Day* columns. After you add the *Date* column, you can set the displayed format. Go to the *Column Tools* in *Data* view, click the *Format* drop-down list, point to *Date Time,* and click 14/*3/2001 dd/MM/yyyy*.



The figure on the right shows the new column after I added it to the dataset, with the applied format.



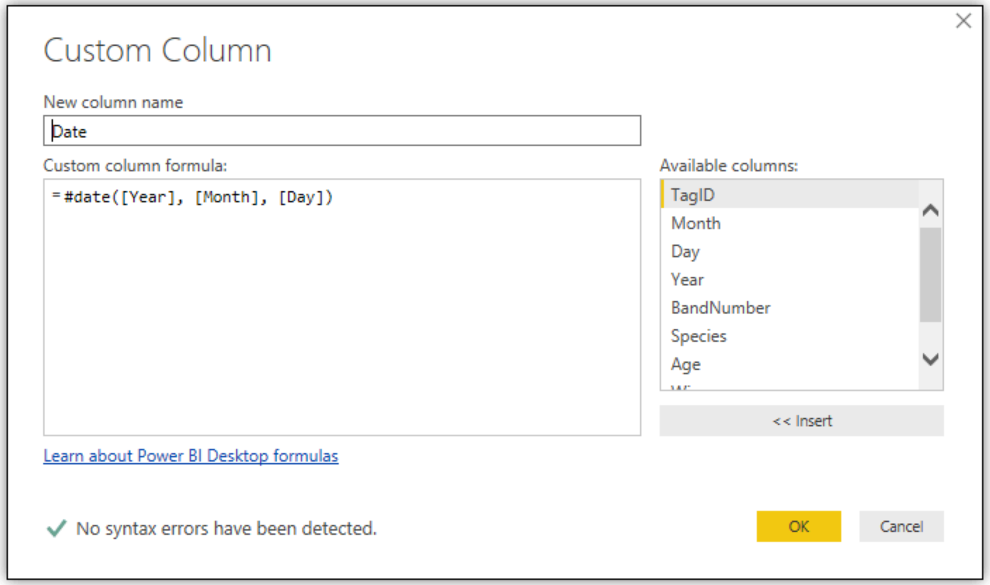
Month	Day	Year	CaptureTime	BandNumber	Species	Age	Wing	Weight	TagID	Date
10	5	1992	13:30	877-76323	RT	I	393	1010	11	05/10/1992
10	8	1992	13:45	877-76324	RT	I	371	1010	12	08/10/1992
10	11	1992	11:00	1207-55912	RT	I	416	1170	15	11/10/1992
10	11	1992	12:40	877-76326	RT	I	418	1150	17	11/10/1992
10	12	1992	13:40	877-76328	RT	I	378	910	19	12/10/1992
10	13	1992	11:00	877-76329	RT	I	396	1010	20	13/10/1992
10	13	1992	14:50	877-76330	RT	I	399	1070	21	13/10/1992
10	14	1992	13:15	877-76331	RT	I	416	1190	22	14/10/1992
10	17	1992	12:10	877-76332	RT	I	415	101	23	17/10/1992

Using DAX to create a column in *Data* view is quick and easy. However, this approach has some limitations. For example, the column is not available in Query Editor, so it cannot be used as part of another calculated column definition in Query Editor. This means that if you open the Query Editor, the Date column will not be there. In addition, if you delete a column referenced by a DAX expression (for example the Day Column), the values in the DAX-based column will show only errors.

The safest way to get around these limitations is to create your column in Query Editor. I will show you how to do this. Delete the *Date* column you just added in the *Data* view. Click *Transform Data* (*Edit Queries)* to reopen the Query Editor. Click the *Custom Column* button on the *Add Column* tab. When the *Custom Column* dialog box appears, type a name for the new column and the column's formula, which is an M expression that defines the column's logic. For example, to create a column like the DAX-based column above, used the following M expression:

= #date([Year], [Month], [Day])

The statement uses the **#date** method to create a date value based on the *Year, Month,* and *Day* columns, similar to the DAX expression. The following figure shows what the M expression looks like when entered into the *Custom Column* dialog box.



When entering your M expression in the *Custom Column* dialog box, be sure that the green arrow is showing at the bottom of the dialog box to ensure that there are no syntax errors. Be aware, however, that you can have errors in your M statement without them showing up as syntax errors.

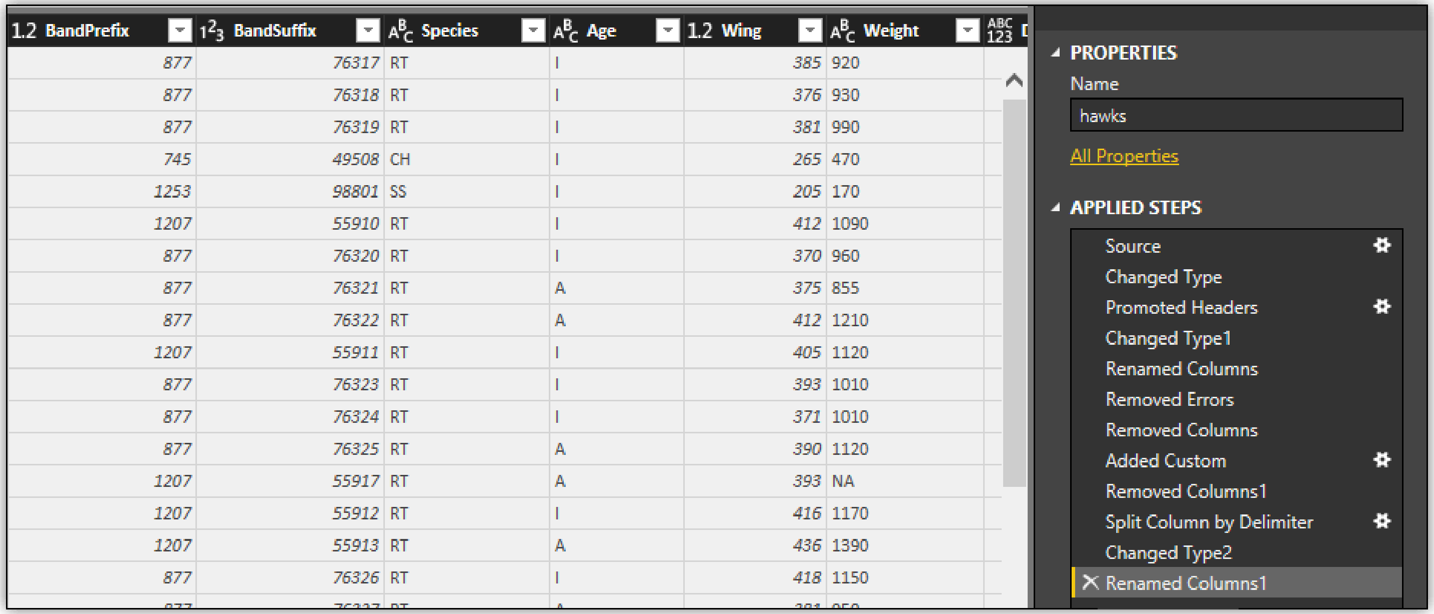
When you create a calculated column, Query Editor adds the *Added Custom* step to the *Applied Steps* section. You can then reference the new column in other calculated columns. You can also remove any of the columns referenced by the M expression. In this case, remove the *Month* and *Day* columns (by right-clicking on them and selecting "Remove") but keep the *Year* column.

Step 6: Splitting a Column

In Query Editor, you can split a column based on a specified value (delimiter) or by a specified number of characters. For example, this dataset includes the *BandNumber* column, which is made up of two parts, separated by a hyphen. One of those parts might have special meaning, such as indicating the individual who tagged the hawk or the process used to tag the hawk. Splitting the column could make it easier to group the data by a particular entity.

To split a column by a delimiter, right-click the column's header, point to *Split Column*, and then click *By Delimiter*. In the *Split Column by Delimiter* dialog box, specify the delimiter and then click *OK*. Query Editor will split the column into two columns (removing the delimiter) and update the data types if necessary. You can then rename the columns if you do not want to use the autogenerated names.

The following figure shows the *hawks* dataset after splitting the *BandNumber* column and changing the names of the two new columns to *BandPrefix* and *BandSuffix.*



Notice that three steps have been added to the *Applied Steps* section: *Split Column by Delimiter, Changed Type2,* and *Renamed Columns1*.

Step 7: Performing Additional Data Transformations

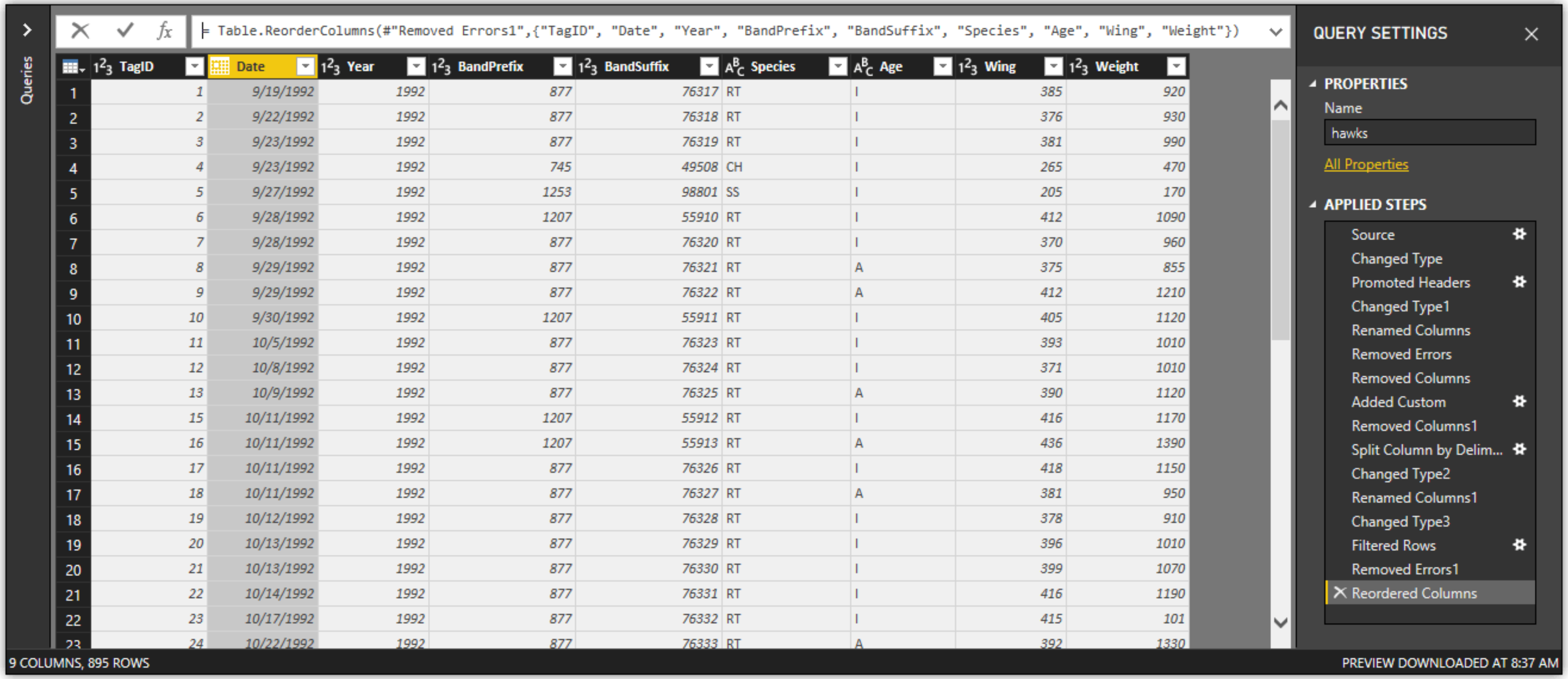
When you're preparing a dataset for reporting, you should evaluate the data types that have been assigned to each column to ensure they're what you need. Usually you'll want to hold off doing this until after you've added, removed, or split columns so you're not introducing unnecessary transformation steps. Evaluating the data types can also point to possible anomalies in the data.

For example, the *BandPrefix (or BandNumber.1)* column was assigned the **number** data type, which would have been expected to be the **Int64** data type. The same goes for the *Wing* column. However, the *Weight* column was assigned the **text** data type, and the *Date* column was assigned the **Any** data type. For this tutorial, change the data types for the *BandPrefix, Wing*, and *Weight* columns to the **Int64** data type (whole number), and change the *Date* column to the **date** data type. If you get any errors, I'll explain what to do in the paragraphs below.

The reason the *Wing* column was assigned the **number** data type was because it contained a decimal value. When changing the data type to **Int64**, Query Editor automatically rounded the value to the nearest whole number, which worked fine for the purposes of this tutorial.

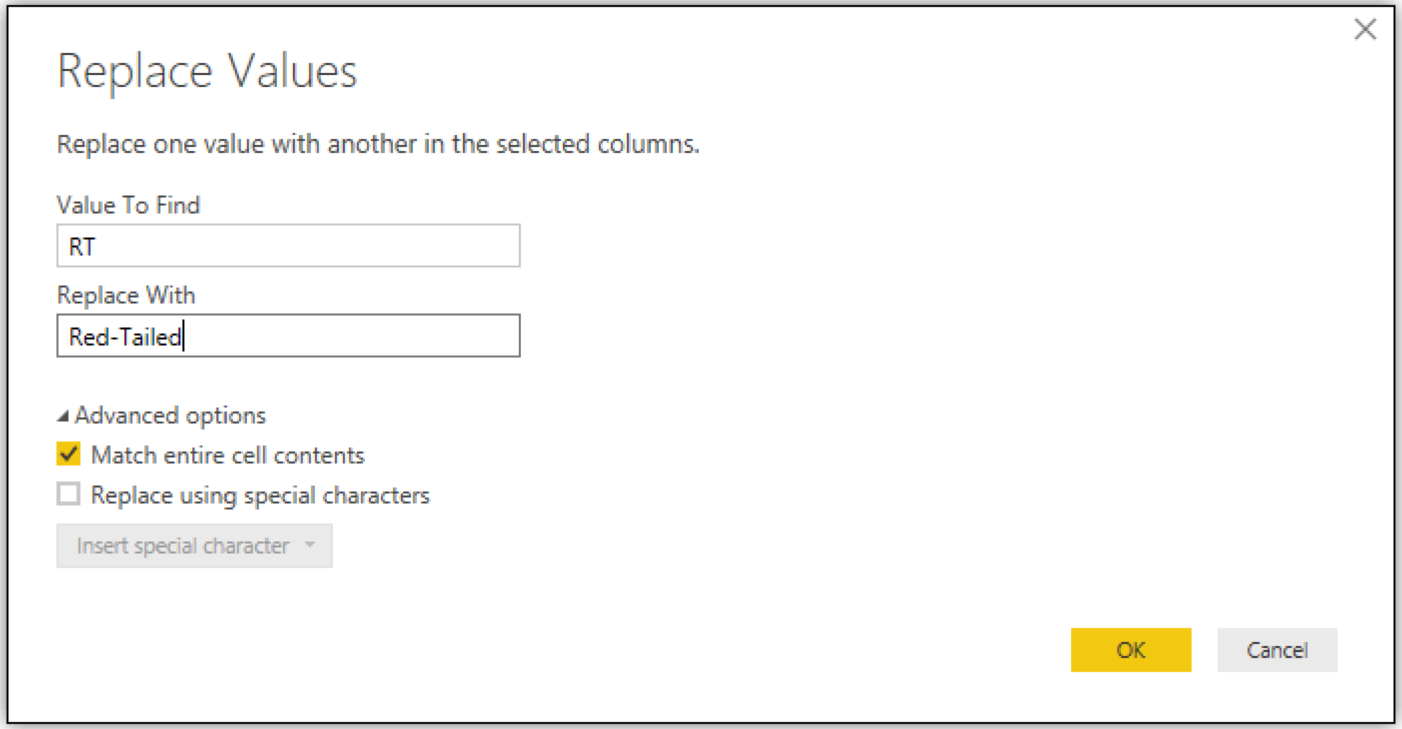
The process is not quite as simple for the *Weight* column. This column was assigned the **text** data type because it contained *NA* values, which were treated as errors when changing the column to the **Int64** data type. Remove these rows by using the *Remove Errors* option (right-click on the column, then click "Remove Errors").

The next step is to move the *Date* column on the left within the dataset so it appears before the *Year* column. To move a column in Query Editor, simply drag the column header to the new position. Query Editor will add the *Reordered Columns* step to the *Applied Steps* section, as shown in the following figure. The figure also shows the other steps that were added when changing data types, filtered rows, and removed errors.



The final step is to replace the values in the *Species* and *Age* columns to make them more readable. To replace a value, right-click the column header, and click *Replace Values*. In the *Replace Values* dialog box, type the original value and new value in the applicable text boxes. Next, click the *Advanced options* arrow, and select the *Match entire cell contents* checkbox. This is important to ensure that you don't replace part of another value.

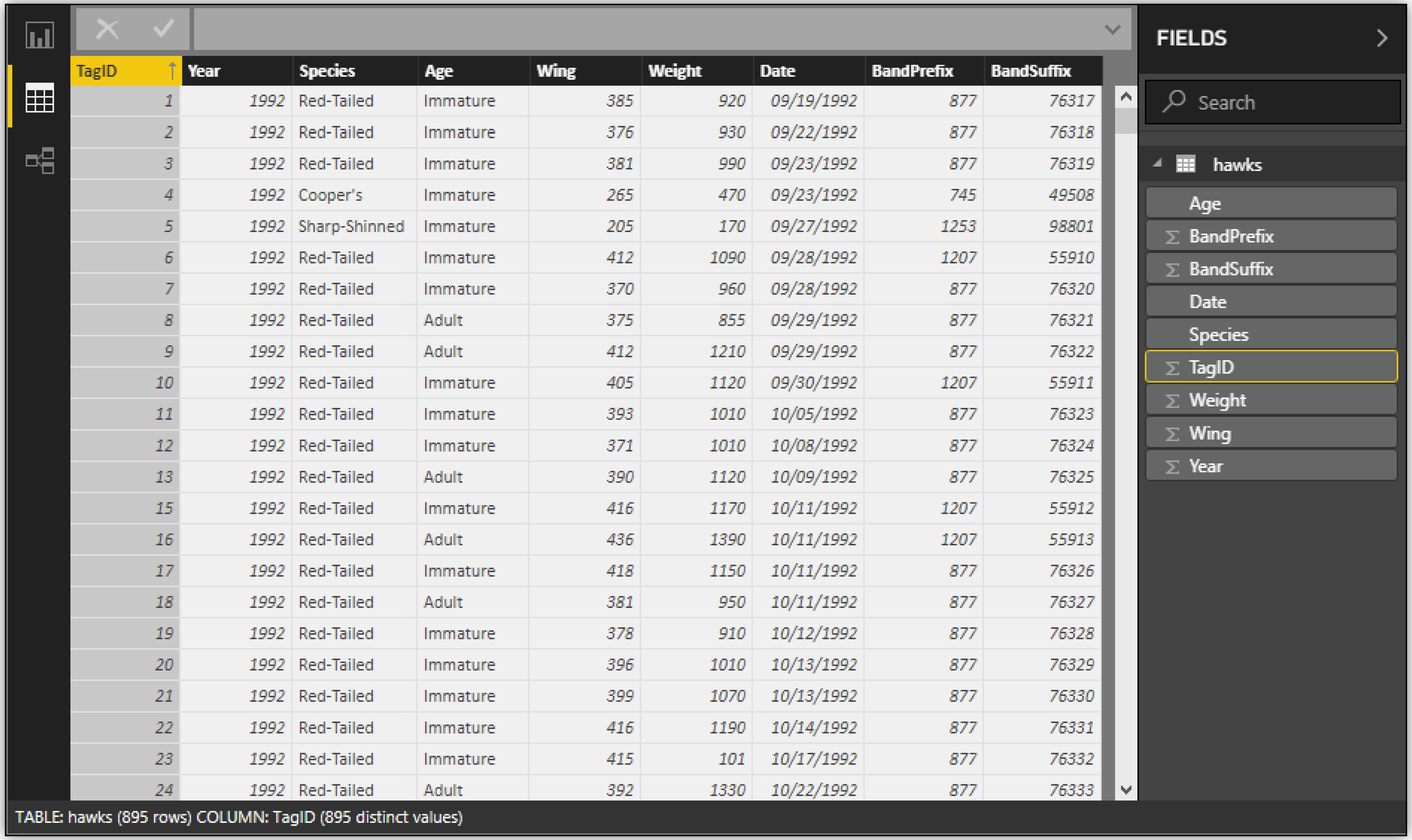
The first value to replace in the *hawks* data set is the *RT* value in the *Species* column. For the new value, used *Red-Tailed,* as shown in the following figure.



Also replace the following values:

- The *CH* value in the *Species* column with *Cooper's*.
- The *SS* value in the *Species* column with *Sharp-Shinned*.
- The *A* value in the *Age* column with *Adult*.
- The *I* value in the *Age* column with *Immature*.

Then click close and apply to go back to main screen. The following figure shows the updated values in the *Species* and *Age* columns, as the dataset appears in the *Data* view of the main Power BI Desktop window.



TagID	Year	Species	Age	Wing	Weight	Date	BandPrefix	BandSuffix
1	1992	Red-Tailed	Immature	385	920	09/19/1992	877	76317
2	1992	Red-Tailed	Immature	376	930	09/22/1992	877	76318
3	1992	Red-Tailed	Immature	381	990	09/23/1992	877	76319
4	1992	Cooper's	Immature	265	470	09/23/1992	745	49508
5	1992	Sharp-Shinned	Immature	205	170	09/27/1992	1253	98801
6	1992	Red-Tailed	Immature	412	1090	09/28/1992	1207	55910
7	1992	Red-Tailed	Immature	370	960	09/28/1992	877	76320
8	1992	Red-Tailed	Adult	375	855	09/29/1992	877	76321
9	1992	Red-Tailed	Adult	412	1210	09/29/1992	877	76322
10	1992	Red-Tailed	Immature	405	1120	09/30/1992	1207	55911
11	1992	Red-Tailed	Immature	393	1010	10/05/1992	877	76323
12	1992	Red-Tailed	Immature	371	1010	10/08/1992	877	76324
13	1992	Red-Tailed	Adult	390	1120	10/09/1992	877	76325
15	1992	Red-Tailed	Immature	416	1170	10/11/1992	1207	55912
16	1992	Red-Tailed	Adult	436	1390	10/11/1992	1207	55913
17	1992	Red-Tailed	Immature	418	1150	10/11/1992	877	76326
18	1992	Red-Tailed	Adult	381	950	10/11/1992	877	76327
19	1992	Red-Tailed	Immature	378	910	10/12/1992	877	76328
20	1992	Red-Tailed	Immature	396	1010	10/13/1992	877	76329
21	1992	Red-Tailed	Immature	399	1070	10/13/1992	877	76330
22	1992	Red-Tailed	Immature	416	1190	10/14/1992	877	76331
23	1992	Red-Tailed	Immature	415	101	10/17/1992	877	76332
24	1992	Red-Tailed	Adult	392	1330	10/22/1992	877	76333

Notice that the columns are not impacted by the reordering that was done in Query Editor. The newer columns—*Date, BandPrefix,* and *BandSuffix*—are included at the right side of the dataset, after the original columns. Also notice that in this case the *Date* column is configured in the format *dd/mm/yyyy,* but you might see the data values spelled out or displayed in another format. However, you can choose any format from the *Format* drop-down list on the *Modeling* ribbon.

Well done! You now have your dataset in the format you want it, you can begin to do data analytics and create reports and their visualizations. We will see how to do this in the next lessons.